

sample_pairs.csv

prompt,reply
hi,hello! how can i help you today?
hello,hi there! what brings you here?
how are you,i'm doing great, thanks for asking!
what is ai,ai is the science of making machines intelligent.
what is machine learning,machine learning lets computers learn patterns from data.
tell me a joke,why did the model cross the road? to optimize the other side!
thanks,you're welcome!
bye,goodbye! have a great day!

requirements.txt

tensorflow>=2.12
pandas
numpy
flask
streamlit

train_gru_chatbot.py

```
#=====
# train_gru_chatbot.py — GRU Seq2Seq Training Script
#=====
# Usage:
# python train_gru_chatbot.py --data_path sample_pairs.csv \
#   --model_dir artifacts --epochs 20 --max_vocab 12000 --max_len 32
#=====
```

```
import os
import re
import json
import argparse
import numpy as np
import pandas as pd
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Embedding, GRU, Dense
```

```
# -----
```

```

# Text cleaning
# -----
START_TOKEN = "<start>"
END_TOKEN = "<end>"

def clean_text(s: str) -> str:
    s = str(s).lower().strip()
    s = re.sub(r"^[a-z0-9?'.! ]+", " ", s)
    s = re.sub(r"\s+", " ", s).strip()
    return s

def load_dataset(path: str) -> pd.DataFrame:
    if not os.path.exists(path):
        data = {
            "prompt": [
                "hi", "hello", "how are you", "what is ai", "what is machine
learning",
                "tell me a joke", "thanks", "bye"
            ],
            "reply": [
                "hello! how can i help you today?",
                "hi there! what brings you here?",
                "i'm doing great, thanks for asking!",
                "ai is the science of making machines intelligent.",
                "machine learning lets computers learn patterns from data.",
                "why did the model cross the road? to optimize the other
side!",
                "you're welcome!",
                "goodbye! have a great day!"
            ]
        }
        df = pd.DataFrame(data)
    else:
        df = pd.read_csv(path)
        df["prompt"] = df["prompt"].apply(clean_text)
        df["reply"] = df["reply"].apply(lambda s: f"{START_TOKEN} " +
clean_text(s) + f" {END_TOKEN}")
    return df

```

```
def vectorize(df: pd.DataFrame, max_vocab: int = 12000, max_len: int = 32):
```

```
    tokenizer = Tokenizer(num_words=max_vocab, oov_token="<oov>")
    tokenizer.fit_on_texts(pd.concat([df["prompt"], df["reply"]], axis=0))
```

```
    enc_in = tokenizer.texts_to_sequences(df["prompt"].tolist())
    enc_in = pad_sequences(enc_in, maxlen=max_len, padding="post")
```

```
    dec_full = tokenizer.texts_to_sequences(df["reply"].tolist())
    dec_in = [seq[:-1] for seq in dec_full]
    dec_tar = [seq[1:] for seq in dec_full]
    dec_in = pad_sequences(dec_in, maxlen=max_len, padding="post")
    dec_tar = pad_sequences(dec_tar, maxlen=max_len, padding="post")
```

```
    vocab_size = min(max_vocab, len(tokenizer.word_index) + 1)
    return enc_in, dec_in, dec_tar, tokenizer, vocab_size
```

```
def build_train_model(vocab_size: int, emb_dim: int, hid_dim: int,
max_len: int):
```

```
    # Shared embedding
```

```
    enc_inputs = Input(shape=(max_len,), name="encoder_inputs")
```

```
    dec_inputs = Input(shape=(max_len,), name="decoder_inputs")
```

```
    emb = Embedding(vocab_size, emb_dim, mask_zero=True,
name="shared_embedding")
```

```
    # Encoder
```

```
    enc_emb = emb(enc_inputs)
```

```
    enc_gru = GRU(hid_dim, return_state=True, name="encoder_gru")
```

```
    _, enc_state = enc_gru(enc_emb)
```

```
    # Decoder (teacher forcing)
```

```
    dec_emb = emb(dec_inputs)
```

```
    dec_gru = GRU(hid_dim, return_sequences=True, return_state=True,
name="decoder_gru")
```

```
    dec_outputs, _ = dec_gru(dec_emb, initial_state=enc_state)
```

```
    dec_dense = Dense(vocab_size, activation='softmax',
name="output_dense")
```

```
    outputs = dec_dense(dec_outputs)
```

```
    model = Model([enc_inputs, dec_inputs], outputs)
```

```
    model.compile(optimizer='adam',
loss='sparse_categorical_crossentropy', metrics=['accuracy'])
```

```
return model
```

```
def save_artifacts(model, tokenizer, model_dir: str):  
    os.makedirs(model_dir, exist_ok=True)  
    model.save(os.path.join(model_dir, "seq2seq.h5"))  
    # Save tokenizer  
    with open(os.path.join(model_dir, "tokenizer.json"), "w", encoding="utf-  
8") as f:  
        f.write(tokenizer.to_json())
```

```
def main():  
    parser = argparse.ArgumentParser()  
    parser.add_argument('--data_path', type=str,  
default='sample_pairs.csv')  
    parser.add_argument('--model_dir', type=str, default='artifacts')  
    parser.add_argument('--epochs', type=int, default=20)  
    parser.add_argument('--batch_size', type=int, default=64)  
    parser.add_argument('--max_vocab', type=int, default=12000)  
    parser.add_argument('--max_len', type=int, default=32)  
    parser.add_argument('--emb_dim', type=int, default=128)  
    parser.add_argument('--hid_dim', type=int, default=256)  
    args = parser.parse_args()  
  
    df = load_dataset(args.data_path)  
    enc_in, dec_in, dec_tar, tokenizer, vocab_size = vectorize(df,  
args.max_vocab, args.max_len)  
  
    model = build_train_model(vocab_size, args.emb_dim, args.hid_dim,  
args.max_len)  
  
    history = model.fit(  
        [enc_in, dec_in],  
        dec_tar[..., None], # sparse targets  
        epochs=args.epochs,  
        batch_size=args.batch_size,  
        validation_split=0.1,  
        verbose=1  
    )  
  
    save_artifacts(model, tokenizer, args.model_dir)
```

```

# Save training meta
meta = {
    "vocab_size": vocab_size,
    "max_len": int(args.max_len),
    "emb_dim": int(args.emb_dim),
    "hid_dim": int(args.hid_dim),
    "start_token": START_TOKEN,
    "end_token": END_TOKEN,
}
with open(os.path.join(args.model_dir, 'meta.json'), 'w') as f:
    json.dump(meta, f, indent=2)

print(f"Saved model + tokenizer to: {args.model_dir}")

```

```

if __name__ == '__main__':
    main()

```

inference_utils.py

```

#=====
# inference_utils.py — Load artifacts and generate replies
#=====
import os
import json
import numpy as np
import re
import tensorflow as tf
from tensorflow.keras.models import load_model
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.preprocessing.text import tokenizer_from_json

START = "<start>"
END = "<end>"

_non_alnum = re.compile(r"[^a-z0-9?'.! ]+")
_multi_space = re.compile(r"\s+")

def clean(s: str) -> str:
    s = str(s).lower().strip()
    s = _non_alnum.sub(" ", s)
    s = _multi_space.sub(" ", s).strip()
    return s

```

```
def load_artifacts(model_dir: str):
    with open(os.path.join(model_dir, 'tokenizer.json'), 'r', encoding='utf-8')
as f:
    tok = tokenizer_from_json(f.read())
    with open(os.path.join(model_dir, 'meta.json'), 'r') as f:
        meta = json.load(f)
    model = load_model(os.path.join(model_dir, 'seq2seq.h5'))
    return model, tok, meta
```

```
def ids_to_word(tokenizer):
    return {idx: w for w, idx in tokenizer.word_index.items()}
```

```
def greedy_decode(model, tokenizer, meta, prompt: str,
max_new_tokens: int = 30):
    max_len = int(meta['max_len'])
    w2i = tokenizer.word_index
    i2w = ids_to_word(tokenizer)

    enc = tokenizer.texts_to_sequences([clean(prompt)])
    enc = pad_sequences(enc, maxlen=max_len, padding='post')

    # decoder starts with <start>
    dec_tokens = [w2i.get(START, 0)]

    for _ in range(max_new_tokens):
        dec_in = pad_sequences([dec_tokens], maxlen=max_len,
padding='post')
        # Model expects [enc_inputs, dec_inputs]
        preds = model.predict([enc, dec_in], verbose=0)
        next_id = int(np.argmax(preds[0, len(dec_tokens)-1])) # last time
step
        if i2w.get(next_id, "") == END:
            break
        dec_tokens.append(next_id)

    words = [i2w.get(i, "") for i in dec_tokens if i not in (0, w2i.get(START, -
1))]
    return " ".join([w for w in words if w]).strip()
```

chatbot_streamlit.py

```
#=====
# chatbot_streamlit.py — Streamlit chat UI
# Run: streamlit run chatbot_streamlit.py
#=====
import streamlit as st
from inference_utils import load_artifacts, greedy_decode

MODEL_DIR = 'artifacts'

@st.cache_resource
def _load():
    model, tok, meta = load_artifacts(MODEL_DIR)
    return model, tok, meta

st.set_page_config(page_title="GRU Chatbot", page_icon="🤖")
st.title("GRU Chatbot")
st.caption("Seq2Seq with GRU — trained on your CSV (prompt,reply)")

model, tok, meta = _load()

if 'history' not in st.session_state:
    st.session_state.history = []

user_input = st.chat_input("Type your message...")

chat = st.container()
with chat:
    for role, text in st.session_state.history:
        st.chat_message(role).markdown(text)

if user_input:
    st.session_state.history.append(("user", user_input))
    st.chat_message("user").markdown(user_input)
    reply = greedy_decode(model, tok, meta, user_input)
    st.session_state.history.append(("assistant", reply))
    st.chat_message("assistant").markdown(reply)
```

chatbot_flask.py

```
=====
# chatbot_flask.py — Flask REST API
# Run: python chatbot_flask.py (http://127.0.0.1:8000)
#=====
```

```
from flask import Flask, request, jsonify
from inference_utils import load_artifacts, greedy_decode
```

```
MODEL_DIR = 'artifacts'
```

```
app = Flask(__name__)
model, tok, meta = load_artifacts(MODEL_DIR)
```

```
@app.route('/health', methods=['GET'])
def health():
    return jsonify({"status": "ok"})
```

```
@app.route('/chat', methods=['POST'])
def chat():
    data = request.get_json(force=True)
    message = data.get('message', "")
    reply = greedy_decode(model, tok, meta, message)
    return jsonify({"reply": reply})
```

```
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8000, debug=False)
```
